Introducción a WebGL (II)

Área de Arquitectura y Tecnología de Computadores – Versión 1.2.12455, 09/02/2026

Índice

1. Introducción
2. Texturas
3. Animación
4. Shaders
   4.1. Shader de vértices
   4.2. Shader de fragmentos
5. Modelos
Entregar

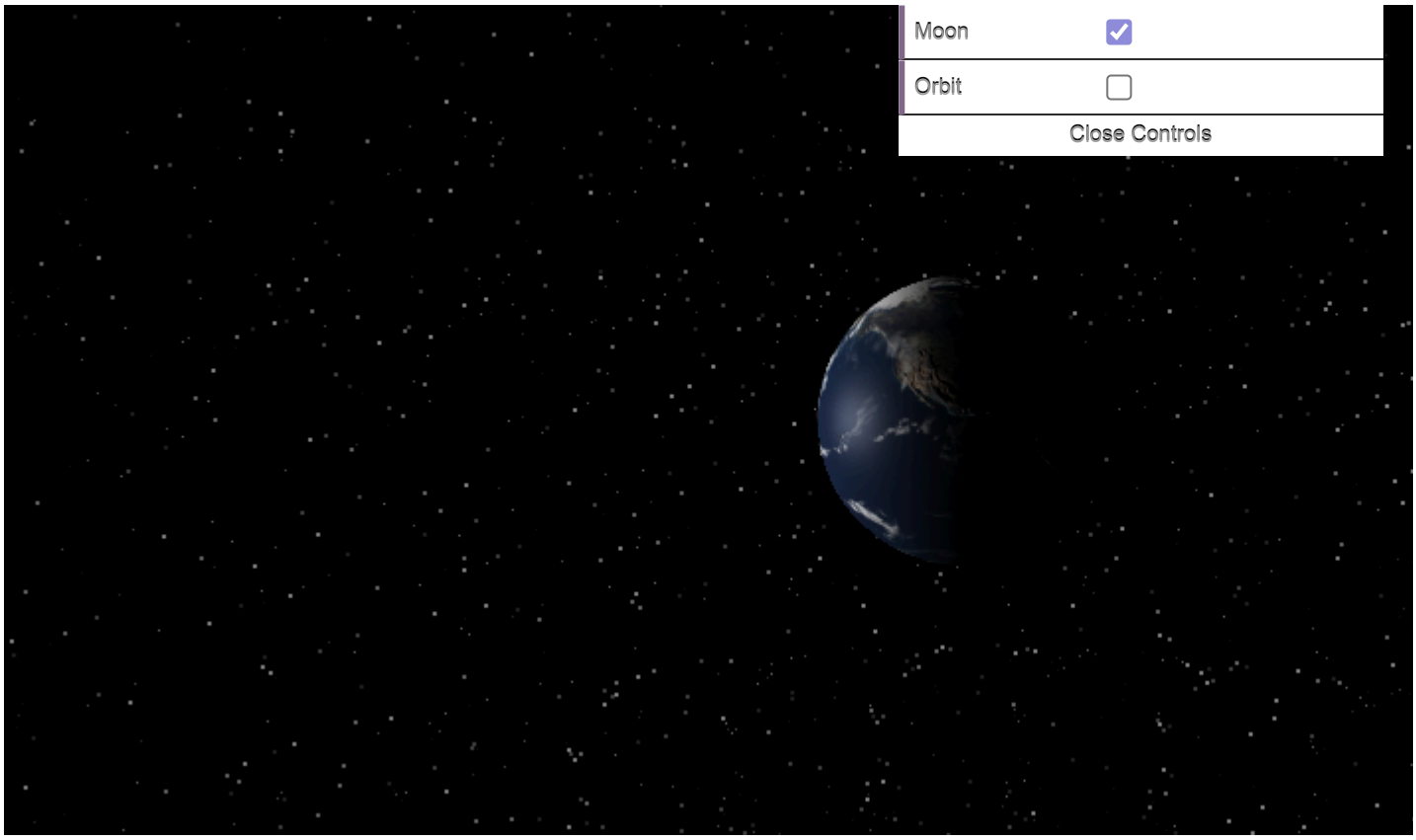
1. Introducción

En la sesión anterior trabajaste con escenas 3D básicas que incluían objetos muy simples. Además, aprendiste a realizar transformaciones sobre los objetos, lo que nos permitía ubicarlos en la escena.

En esta sesión utilizaremos texturas en la definición de los objetos para darles más realismo, para luego animarlos y permitir cierta interacción con el usuario. Finalmente, aprenderemos cómo importar modelos de objetos desarrollados con programas de modelado 3D.

Para ilustrar los conceptos de esta sesión práctica desarrollaremos una escena como la mostrada a continuación, que representa el sistema formado por la Tierra y la Luna. En este escenario el Sol se encuentra a la izquierda y no cambia su posición con respecto al sistema Tierra-Luna.

Modelo del sistema Tierra-Luna



- Obviamente la escena contiene gran cantidad de incorrecciones respecto al mundo real. La más importante es que la distancia a la que orbita la Luna alrededor de la Tierra no está a escala, sino que orbita mucho más cerca para que pueda verse todo el modelo en la escena. El resto de medidas (tamaño de la Tierra y la Luna, inclinación de los planos de órbita y giro, etc.) sí están a escala.

Esta escena está compuesta por varios objetos formando una jerarquía. Como has visto en la sesión anterior, la disposición de objetos en una jerarquía permite realizar transformaciones lineales (traslación, escalado, rotación) sobre un objeto y todos sus descendientes. De esta forma, es posible rotar, escalar o trasladar un sistema compuesto de varios objetos y al mismo tiempo permitir transformaciones sobre los propios objetos que forman el sistema. La jerarquía de esta escena es la que se muestra a continuación.

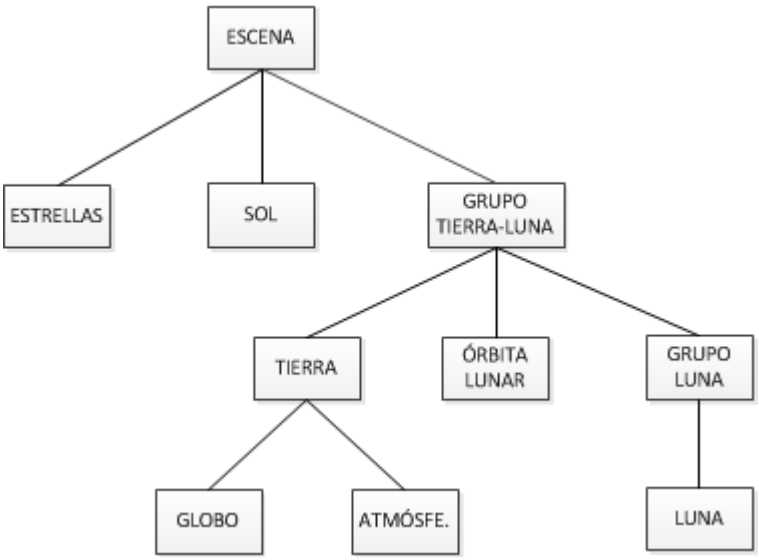


Figura 1. Jerarquía de objetos de la escena

Veamos paso a paso cómo generar la escena anterior. Pero antes de comenzar:

- Recuerda sincronizar tu repositorio local con el repositorio de Bitbucket.
- Crea un subdirectorio para esta práctica de nombre `sesion2` en el directorio que contiene el repositorio git.
- Inicializa un nuevo proyecto para esta sesión, tal como se ilustró en la sesión anterior.

2. Texturas

Una textura es un patrón o una imagen que se utiliza para modificar las características de una superficie. El uso de texturas con three.js es muy sencillo. Vamos a realizar la siguiente escena:

Cubo texturizado

Vamos a ver la parte del código más importante para construir la escena. La textura a utilizar es la de la siguiente figura.

Figura 2. Textura a utilizar con el cubo

- Crea una copia de la página web `prac1-3.html` desarrollada en la sesión anterior y guárdala como `prac2-1.html`.
- Crea también su correspondiente fichero `src/prac2-1.js` y añádelo como punto de entrada en la configuración de webpack.
- Guarda la textura en un subdirectorio de nombre `textures` dentro de `sesion2`.
- Para utilizar una textura debe especificarse la opción `map` cuando se crea el material del objeto. Modifica la escena para especificar la textura al crear el material del cubo mediante el siguiente código.

JAVASCRIPT

```javascript
const mapUrl = "../textures/crate.gif";   // The file used as texture
const textureLoader = new THREE.TextureLoader( );  // The object used to load textures
const map = textureLoader.load( mapUrl );
const material = new THREE.MeshBasicMaterial( { map: map } );
```

Observa que la variable `mapUrl` indica la URL de la textura. En este caso es un fichero ubicado en el mismo subdirectorio `textures`.

- Comprueba el resultado. Dependiendo del navegador, es posible que el cubo se vea negro. Accede a la consola de desarrollador del navegador con `Ctrl` + `Mays` + `i`. Verás que no aparece ningún error. ¿Entiendes qué ha pasado?

La razón es que la carga de las texturas se hace de forma asíncrona mediante peticiones HTTP adicionales al servidor. Esto implica que podría darse el caso de que la escena hubiese terminado de renderizarse antes de que la textura se hubiese descargado completamente (especialmente con texturas grandes).

- Accede a la documentación de la función TextureLoader.load() (https://threejs.org/docs/#api/en/loaders/TextureLoader.load).

Verás que la función recibe más parámetros además de la URL de la textura. Se trata de funciones de *callback* que se irán llamando según avanza la petición HTTP asíncrona. En nuestro caso, por simplicidad, podemos definir únicamente el *callback* para cuando la descarga esté completada para volver a renderizar la escena.

- Añade una función de *callback* que renderice la escena cuando acabe de cargarse la textura. Puedes utilizar el siguiente código para la función.

JAVASCRIPT

```javascript
( loaded ) => { renderer.render( scene, camera ); }
```

- Comprueba que ahora ya sí se ve el cubo texturizado correctamente.

Vamos ahora a construir el modelo Tierra-Luna inicialmente planteado.

- Crea una página de nombre `prac2-2.html` donde aparezca una esfera y una luz de tipo `PointLight` que incida de forma lateral sobre la esfera. Este punto de luz emulará nuestro sol, que se encontrará fuera de la escena. El material de la esfera debe ser de tipo `MeshPhongMaterial`, ya que tendrá componentes ambiental, difusa y especular.
- Añade la siguiente textura al material:

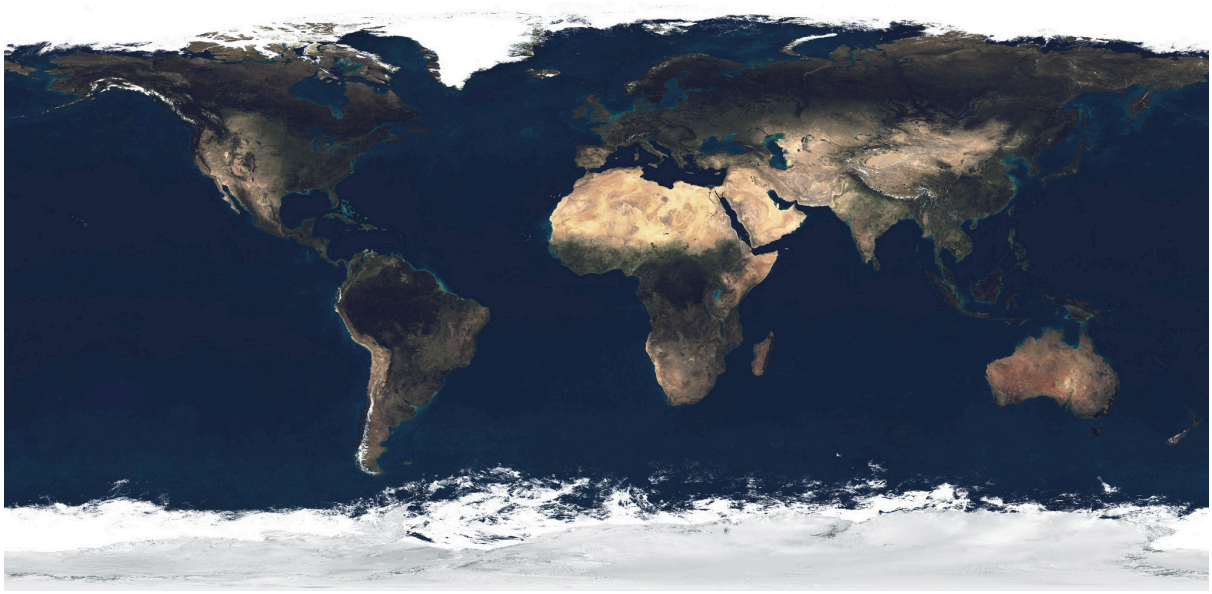


Figura 3. Textura para la superficie terrestre

- Comprueba que la escena se parece todavía poco a la escena final, pero el aspecto de la Tierra es bastante realista gracias al uso de la textura.

> Es posible que tengas que modificar los parámetros de configuración de la luz para que tenga la suficiente intensidad. Una luz de tipo `PointLight` (https://threejs.org/docs/#api/en/lights/PointLight) tiene un decaimiento por defecto con la distancia.

Ahora añadiremos la atmósfera al modelo. Para ello utilizaremos una esfera ligeramente mayor que la utilizada para representar el globlo terráqueo.

- Crea la escena de nombre `prac2-3.html` como copia de `prac2-2.html` y añade una esfera a modo de atmósfera. Esta esfera debe ser transparente para dejar ver lo que hay en la superficie del planeta, pero a la vez debe mostrar algunas nubes. Las nubes no producen reflejos especulares de la luz solar, por lo que el material de la atmósfera debe ser de tipo `MeshLambertMaterial`.
- Añade la siguiente textura al material:

Figura 4. Textura para la atmósfera terrestre

> Aunque no lo puedas apreciar por el formato de este guion, la imagen es transparente y contiene nubes blancas.

- Comprueba el resultado. Verás las nubes pero no el terreno. El problema es que la textura no es transparente. Para que la atmósfera sea transparente debe indicarse la transparencia al crear el material con el siguiente código. Este código define el material de la atmósfera como transparente y blanco en aquellas partes opacas (las nubes).

JAVASCRIPT
```
var atmosphereMaterial = new THREE.MeshLambertMaterial( { color: 0xFFFFFF, map: atmosphereMap,
transparent: true } );
```

Por otro lado, el eje de rotación de la tierra está inclinado respecto al plano que recorre su órbita alrededor del Sol aproximadamente 23 grados.

> Puedes consultar este (inclinación axial) y otros datos sobre la Tierra en su entrada de la Wikipedia (https://es.wikipedia.org/wiki/Tierra).

En nuestro modelo el plano de la órbita terrestre alrededor del sol será el plano Z (z=0). Por tanto, habrá que introducir una rotación sobre el eje Z de 0.36 radianes (23 grados). Ten en cuenta que esta rotación afecta tanto al globo terráqueo como a la atmósfera, por lo que será necesario un `Object3D` que incluya ambos elementos y realizar la rotación sobre el objeto para que afecte al mismo tiempo al globo y la atmósfera.

- Crea una escena de nombre `prac2-4.html` como copia de `prac2-3.html`.
- Rota la Tierra sobre el plano de la órbita en esta nueva escena.

> Por simplicidad nuestro modelo no considera el bamboleo en el eje de rotación terrestre.

Ahora vamos con la Luna.

- Crea la escena de nombre `prac2-5.html` como copia de `prac2-4.html`.
- En esta nueva escena, fija un tamaño para la Luna a escala con la Tierra. Lo más cómodo es asignarle a la Tierra un tamaño de 1 y a la Luna un tamaño que corresponda a la fracción del tamaño de la Tierra correspondiente (0.27 tierras).
- Añade la siguiente textura a la Luna.

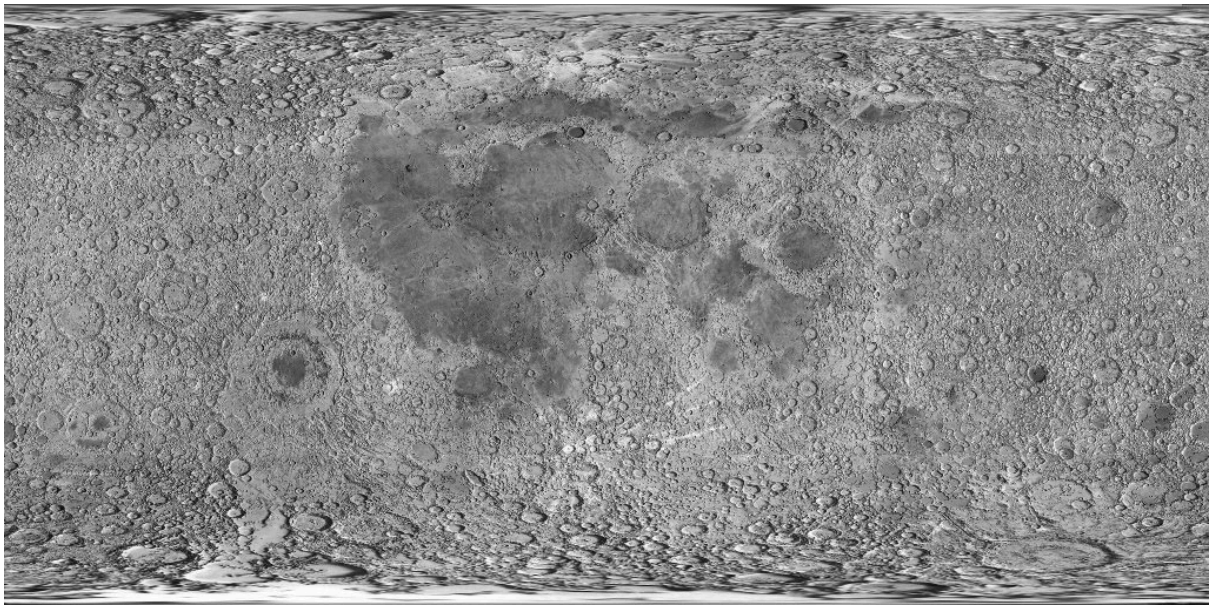


Figura 5. Textura para la superficie lunar

- Añade la Luna a la escena. A continuación se muestra un trozo de código que te puede servir de ayuda.

JAVASCRIPT

```
const moonMapUrl = 'moon_1024.jpg';
const moonMap = textureLoader.load( moonMapUrl, ( loaded ) => { renderer.render( scene, camera
); } );
const material = new THREE.MeshLambertMaterial( { map: moonMap, color: 0x888888 } );

//... TODO: create the Moon and compute the distance to the Earth

// Move the Moon away from the coordinate origin (the Earth)
// NOT TO SCALE. Real value: Math.sqrt( distance * distance / 2 )
moon.position.set( Math.sqrt( distance / 2 ), 0, -Math.sqrt( distance / 2 ) );

// Rotate the Moon to face visible side to the Earth (tidal locking)
moon.rotation.y = Math.PI;

// Moon should rotate around the Earth: an Object3D is needed
const moonGroup = new THREE.Object3D( );
moonGroup.add( moon );

// The Moon orbit is a bit tilted
moonGroup.rotation.x = 0.089;
```

3. Animación

Animar una escena consiste en hacer que varíe con el tiempo. Para animar una escena habitualmente utilizaremos un código como el siguiente:

JAVASCRIPT

```javascript
const clock = new THREE.Clock( );

function animate( ) {

    const delta = clock.getDelta( ); // Elapsed time in seconds

    // UPDATE THE SCENE ACCORDING TO THE ELAPSED TIME

    // Render the scene
    renderer.render( scene, camera );

    // Request the browser to execute the animation-rendering loop
    requestAnimationFrame( animate );
};
```

Gracias a este marco de trabajo, es fácil implementar animaciones simples como la que vamos a realizar sobre nuestro sistema Tierra-Luna.

- Crea una escena de nombre `prac2-6.html` como copia de `prac2-5.html`.
- Anima el giro de la Tierra en esta nueva escena. Puedes utilizar el siguiente código combinado con el anterior.

JAVASCRIPT

```javascript
// UPDATE THE SCENE ACCORDING TO THE ELAPSED TIME
const rotation = ( delta * Math.PI * 2 ) / 24;
earthGlobe.rotation.y += rotation;
atmosphere.rotation.y += rotation * 0.95;
```

El código anterior considera un tiempo de rotación de la Tierra de 24 segundos y hace que el globo terráqueo y la atmósfera roten a diferentes velocidades para simular el movimiento de las nubes.

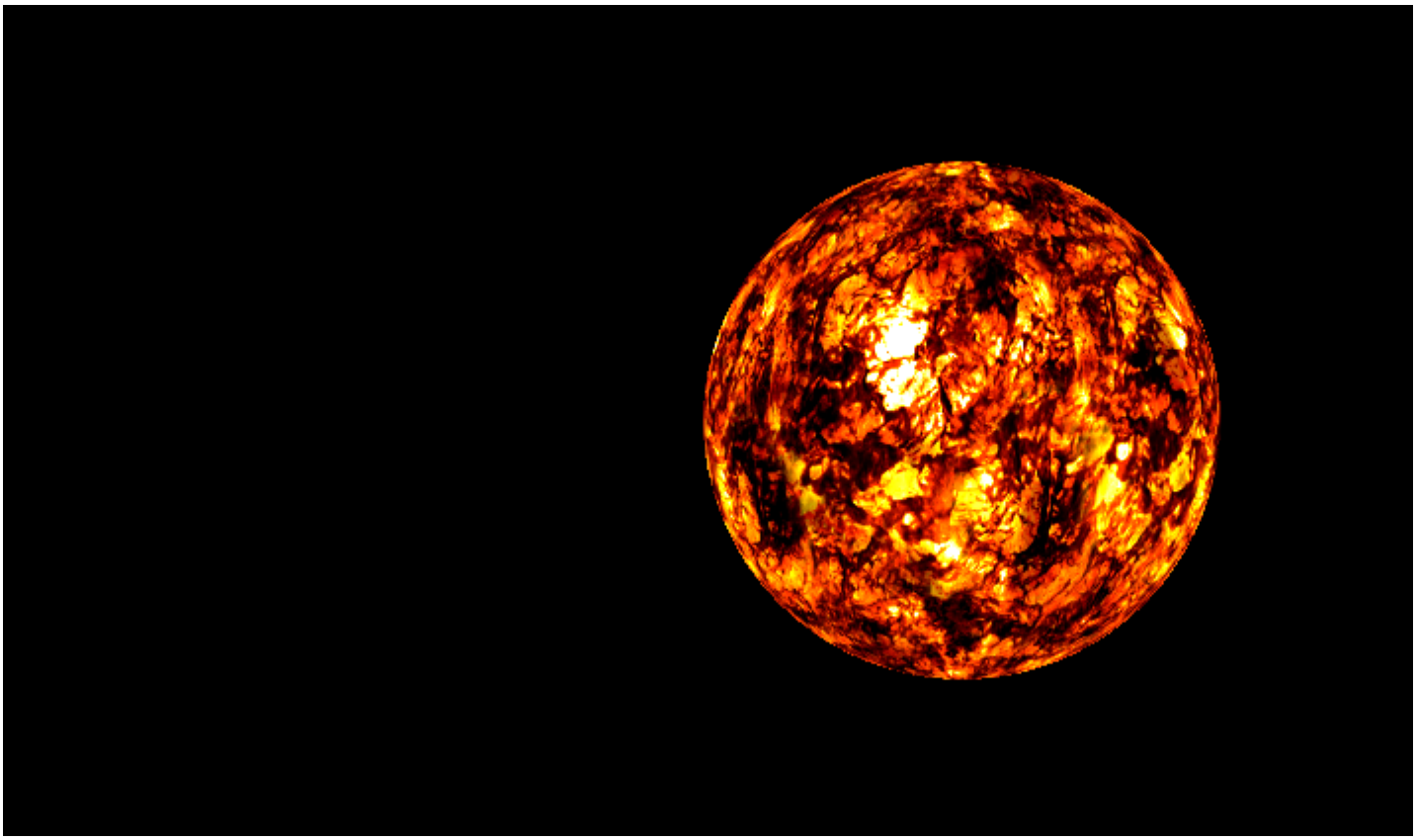
- Crea una escena de nombre `prac2-7.html` como copia de `prac2-6.html`.
- Anima la rotación de la Luna alrededor de la Tierra en la nueva escena. Recuerda que la Luna completa su órbita cada 28 días.

4. Shaders

En ocasiones, el uso de materiales y texturas no es suficiente para representar objetos de forma realista. Hoy en día, OpenGL, y por extensión WebGL, ofrece la posibilidad de programar etapas en el *pipeline* gráfico para introducir efectos en la renderización de los gráficos. Esto se consigue con pequeños programas llamados *shaders* escritos en un lenguaje específico, denominado GLSL (https://www.khronos.org/registry/OpenGL/specs/gl/GLSLangSpec.4.60.pdf), muy parecido a C. Puedes consultar un tutorial de GLSL (http://www.lighthouse3d.com/tutorials/glsl-tutorial/) para más información.

Existen varios tipos de *shaders*. Los más habituales son los *shaders* de vértices y los de fragmentos. A modo de ejemplo, utilizaremos un *shader* de vértices y otro de fragmentos para nuestro sol, que dará el resultado mostrado en la siguiente escena.

Modelo del Sol utilizando shaders



No es objetivo de la asignatura programar *shaders*, pero sí ver cómo pueden utilizarse en nuestras escenas.

Incrustaremos los *shaders* en nuestras escenas utilizando un cargador específico de webpack, por lo que es necesario modificar la configuración de webpack del proyecto.

- Instala el cargador de webpack para ficheros GLSL.

```
$> npm install webpack-glsl-loader -D
```

- Modifica el fichero `webpack.config.js` para añadir la configuración del cargador al objeto `module.exports`.

JAVASCRIPT

```
module: {
    rules: [
      {
        test: /\.glsl$/,
        use: {
          loader: 'webpack-glsl-loader'
        }
      }
    ]
  }
```

Para utilizar *shaders* en threejs es necesario utilizar materiales de tipo `ShaderMaterial`, pasándole los *shaders* a utilizar y un objeto que contiene variables a utilizar dentro de estos. Este objeto recibe el nombre de `uniforms`. Concretamente, para nuestro sol definiremos este objeto de la siguiente manera:

JAVASCRIPT

```javascript
const NOISEMAP = 'cloud.png';
const SUNMAP = 'lavatile.jpg';
const textureLoader = new THREE.TextureLoader( );
const uniforms = {
    "fogDensity": { value: 0 },
    "fogColor": { value: new THREE.Vector3( 0, 0, 0 ) },
    "time": { value: 1.0 },
    "uvScale": { value: new THREE.Vector2( 3.0, 1.0 ) },
    "texture1": { value: textureLoader.load( NOISEMAP ) },
    "texture2": { value: textureLoader.load( SUNMAP ) }
};

uniforms[ "texture1" ].value.wrapS = uniforms[ "texture1" ].value.wrapT = THREE.RepeatWrapping;
uniforms[ "texture2" ].value.wrapS = uniforms[ "texture2" ].value.wrapT = THREE.RepeatWrapping;
```

A través de este objeto le pasaremos una serie de parámetros a los *shaders*:

- *fogDensity*: Define cómo de densa (transparencia) será la niebla que se desplegará sobre la superficie.
- *fogColor*: Define el color de la niebla.
- *time*: Permite variación con el tiempo.
- *uvScale*: Relación entre las coordenadas uv (textura).
- *texture1*: Textura que representa ruido sobre la textura principal. Utilizaremos la siguiente imagen:

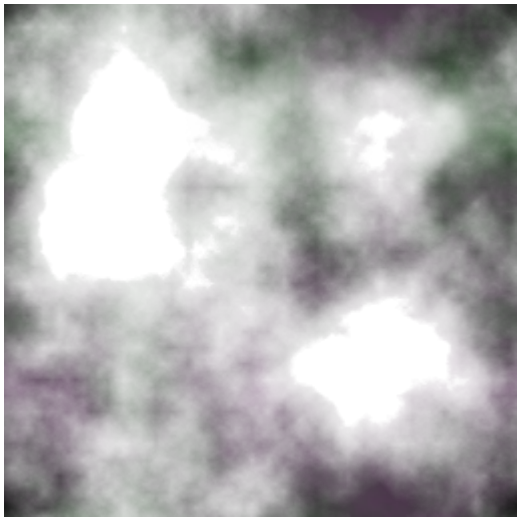


Figura 6. Textura para simular ruido en la superficie solar

- *texture2*: Textura principal de la superficie del Sol. Utilizaremos esta imagen:

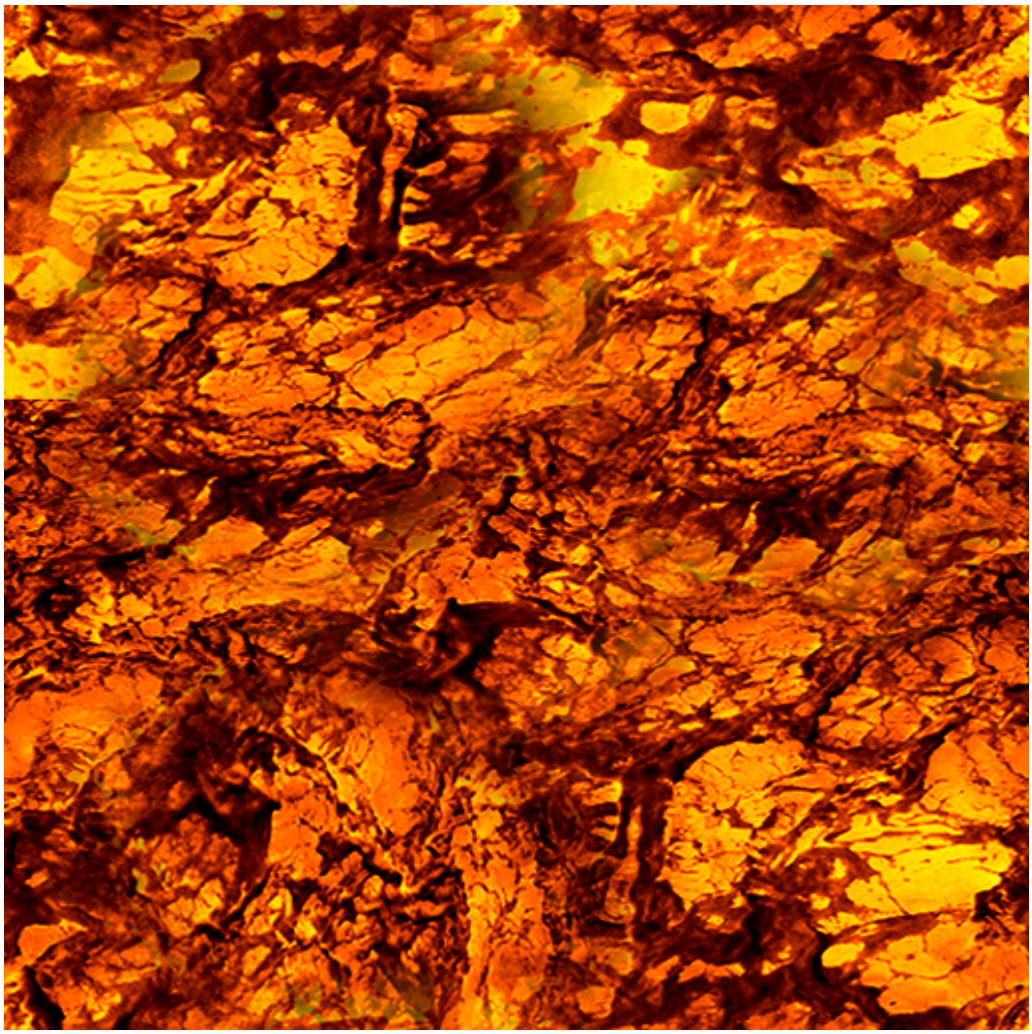


Figura 7. Textura para simular la superficie solar

Una vez creado el objeto `uniforms`, puede crearse el material de tipo `ShaderMaterial` tal como se indica a continuación.

JAVASCRIPT

```
const vertexShader = require( 'path to the vertex shader' );
const fragmentShader = require( 'path to the fragment shader' );

const material = new THREE.ShaderMaterial( {
    uniforms,
    vertexShader,
    fragmentShader
} );
```

4.1. Shader de vértices

Un *shader* de vértices (*vertex shader*) es un programa escrito en GLSL que se ejecuta en la GPU y trabaja sobre un vértice individual, generando como salida otro vértice. Este nuevo vértice puede ser el mismo o una transformación del mismo. Este *shader* no tiene información acerca de otros vértices, solo los siguientes datos (entre otros) del vértice a procesar: posición, vector normal y coordenadas en la textura a mapear.

- Crea una nueva escena de nombre `prac2-8.html` como copia de la escena `prac2-7.html`.
- Crea un fichero de texto **con codificación ANSI** de nombre `shaders/vertex.glsl` con el siguiente contenido.

GLSL

```
uniform vec2 uvScale;
varying vec2 vUv;

void main( ) {

    vUv = uvScale * uv;
    vec4 mvPosition = modelViewMatrix * vec4( position, 1.0 );
    gl_Position = projectionMatrix * mvPosition;

}
```

- Crea la esfera solar en el mismo punto donde se encuentra la luz de la escena. Dimensiona la esfera como 5 veces el tamaño de la Tierra.
- Referencia este *shader* cuando crees el material de la esfera solar.

4.2. Shader de fragmentos

Un *shader* de fragmentos (*fragment shader*) es de nuevo un programa escrito en GLSL que se ejecuta en la GPU, en este caso tras la etapa de rasterización, y trabaja sobre un único fragmento. Como ocurría con el *vertex shader*, el *fragment shader* únicamente tiene acceso a la información del fragmento a procesar, que incluye la posición (X,Y) del píxel y todos los atributos interpolados en la etapa de rasterización.

- Crea un fichero de texto **con codificación ANSI** de nombre `shaders/fragment.glsl` con el siguiente contenido.

GLSL

```
uniform float time;

uniform float fogDensity;
uniform vec3 fogColor;

uniform sampler2D texture1;
uniform sampler2D texture2;

varying vec2 vUv;

void main( void ) {

    vec2 position = - 1.0 + 2.0 * vUv;

    vec4 noise = texture2D( texture1, vUv );
    vec2 T1 = vUv + vec2( 1.5, - 1.5 ) * time * 0.02;
    vec2 T2 = vUv + vec2( - 0.5, 2.0 ) * time * 0.01;

    T1.x += noise.x * 2.0;
    T1.y += noise.y * 2.0;
    T2.x -= noise.y * 0.2;
    T2.y += noise.z * 0.2;

    float p = texture2D( texture1, T1 * 2.0 ).a;

    vec4 color = texture2D( texture2, T2 * 2.0 );
    vec4 temp = color * ( vec4( p, p, p, p ) * 2.0 ) + ( color * color - 0.1 );

    if( temp.r > 1.0 ) { temp.bg += clamp( temp.r - 2.0, 0.0, 100.0 ); }
    if( temp.g > 1.0 ) { temp.rb += temp.g - 1.0; }
    if( temp.b > 1.0 ) { temp.rg += temp.b - 1.0; }

    gl_FragColor = temp;

    float depth = gl_FragCoord.z / gl_FragCoord.w;
    const float LOG2 = 1.442695;
    float fogFactor = exp2( - fogDensity * fogDensity * depth * depth * LOG2 );
    fogFactor = 1.0 - clamp( fogFactor, 0.0, 1.0 );

    gl_FragColor = mix( gl_FragColor, vec4( fogColor, gl_FragColor.w ), fogFactor );
}
```

- Referencia este *shader* cuando crees el material de la esfera solar.
- Anima la textura con el siguiente código (se asume que la variable `uniforms` es accesible desde la función de animación.

JAVASCRIPT

```
uniforms[ "time" ].value += 0.2 * delta;
```

- Opcionalmente puedes rotar el Sol apropiadamente dentro de la función de animación.
- Prueba la escena.

5. Modelos

En escenas complejas es habitual importar modelos desarrollados con aplicaciones de modelado 3D para evitar la complejidad de definir objetos por programación. La biblioteca three.js admite varios formatos de importación, si bien los más comunes son COLLADA y JSON.

- Échale un vistazo a los ejemplos de three.js (http://threejs.org/examples/) para ver todas las posibilidades.

Vamos a importar un modelo de la estación espacial internacional que podríamos incorporarlo a nuestra escena. Este modelo está en formato COLLADA, por lo que necesitaremos utilizar la extensión de three.js para cargar modelos COLLADA.

Modelo de la ISS

- Crea una nueva escena de nombre `prac2-9.html` .
- Descarga el modelo COLLADA de la estación espacial internacional y descomprímelo en el subdirectorio `models` dentro de `sesion2` .
- Importa el cargador de modelos COLLADA con el siguiente código.

JAVASCRIPT

```javascript
import { ColladaLoader } from 'three/examples/jsm/loaders/ColladaLoader.js';
```

- Añade el siguiente código para importar el modelo a tu escena.

JAVASCRIPT

```javascript
const modelUrl = "../models/iss.dae";
let iss;

const loadingManager = new THREE.LoadingManager( ( ) => {

    scene.add( iss );
    console.log( 'Model loaded' );
} );

const loader = new ColladaLoader( loadingManager );
loader.load( modelUrl, ( collada ) => {

    iss = collada.scene;
    iss.scale.x = iss.scale.y = iss.scale.z = 0.3;
    iss.rotation.set( Math.PI / 5, Math.PI / 5, 0 );
    iss.updateMatrix( );
} );
```

- El código anterior carga el modelo y lo reescala (habitualmente habrá que reescalar los modelos COLLADA). Fíjate cómo el modelo se carga de forma asíncrona, así que se le pasa a la función `load` una función de *callback* a invocar cuando finalice la carga del modelo.

> Puedes acceder a la galería 3D de SketchUp (https://3dwarehouse.sketchup.com/) donde encontrarás infinidad de modelos.

Entregar

Al finalizar la sesión se creará una página web (fichero html) dentro del proyecto a modo de memoria utilizando la plantilla de prácticas cubriendo los siguientes aspectos:

- Un resumen de las tareas realizadas en la sesión y los resultados obtenidos, detallando las dificultades surgidas durante su desarrollo y la forma de solucionarlas.
- TODAS las escenas generadas (cada una incluida dentro del objeto *canvas* correspondiente) y respuestas concisas correspondientes a las cuestiones planteadas en el guión.
- Las capturas de pantalla que consideres más significativas si hubiere lugar.

El repositorio git ya con la memoria incluida se exportará a un fichero ZIP y se subirá a la tarea correspondiente en la página web de la asignatura. El fichero zip deberá contener los siguientes ficheros:

- index.html (siguiendo la plantilla dada).
- report.css

- prac2-1.html a prac2-9.html
- style.css
- package.json
- package-lock.json
- webpack.config.js
- src/
  - prac2-1.js a prac2-9.js
- .vscode/
  - launch.js
- models/
  - iss.dae
- images/
  - todas las imagenes del modelo cargado
- textures/
  - todas las texturas utilizadas
- shaders/
  - fragment.glsl
  - vertex.glsl